

Deep Learning & NN 1 - Retake - March 2025

Under which assumption does the optimization procedure for the Perceptron converge?

Select one or more:

- ☐ a. It converges for all problems as long as the data is normalized
- ☐ b. It is never guaranteed to converge
- ☐ c. It converges only if the data is in range $[0, 1]$ and linearly separable
- ☐ d. It converges for linearly separable problems after finite many steps
- ☐ e. It converges for all problems after finite many steps

What is the objective function of the perceptron?

Select one:

- ☐ a. $-\sum_{n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☐ b. $-\sum_{n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☐ c. $-\sum_{n=1}^N a^n y^n \mathbf{w}^T \mathbf{x}^n$ where $a^n = \text{sign}(\mathbf{w}^T \mathbf{x}^n)$
- ☐ d. $-\sum_{n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☐ e. $-\sum_{n=1}^N a^n y^n \mathbf{w}^T \mathbf{x}^n$ where $a^n = \text{sign}(\mathbf{w}^T \mathbf{x}^n)$

3) What is the activation function of the Perceptron?

- step or sign
- Linear
- ReLU
- ...

4) MC on softmax regression

- Vector from 1 to K
- vector of probabilities that add up to 1
- unlike logistic regression, softmax has a closed form
- ...

Write down the recursion formula for delta-errors of the backpropagation algorithm using the notation of the lecture.

$$\delta^{l-1} = \dots$$

Wählen Sie eine Antwort:

☐ a.

$$\delta^{l-1} = \delta^l \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1})) f(s^{l-1})$$

☐ b.

$$\delta^{l-1} = \delta^{l-1} \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1}))$$

☒ c.

$$\delta^{l-1} = \delta^l \mathbf{W}^l \text{diag}(f'(s^l))$$

☐ d.

$$\delta^{l-1} = \delta^l \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1})) s^{l-1}$$

☐ e.

$$\delta^{l-1} = \delta^l \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1}))$$

The Backpropagation algorithm of a Deep Neural Network with loss $L(\mathbf{y}, \hat{\mathbf{y}})$ consisting of L layers w.r.t. the weights $w_{ij}^{[1]}$ in the first layer is given as:

Wählen Sie eine Antwort:

☐ a.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}} \dots \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} \dots \frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}$$

☐ b.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{s}^{[L]}} \dots \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} \dots \frac{\partial \mathbf{s}^{[2]}}{\partial w_{ij}^{[1]}}$$

☐ c.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}} \dots \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} \dots \frac{\partial \mathbf{s}^{[2]}}{\partial w_{ij}^{[1]}}$$

☐ d.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}} \dots \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l]}} \dots \frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}$$

☐ e.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}} \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} \dots \frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}$$

☐ f.

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{s}^{[L]}} \dots \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} \dots \frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}$$

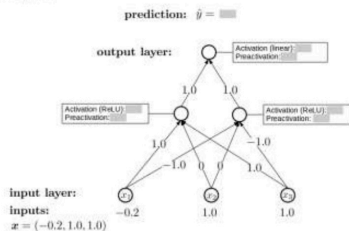
For training a machine learning model $g(\mathbf{x}; \mathbf{w})$ with stochastic gradient descent, the important quantity to obtain updates is the gradient of the loss function $\mathcal{L}$ with respect to the inputs $\mathbf{x} : \nabla_{\mathbf{x}} \mathcal{L}(y, g(\mathbf{x}; \mathbf{w}))$, True or False?

Wählen Sie eine Antwort:

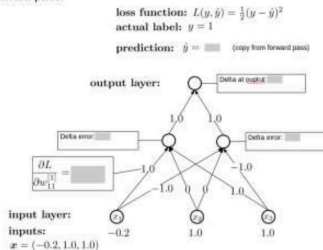
- ☐ a. True, the gradient w.r.t. the inputs determine the direction in which to go on the loss function to minimize the loss
- ☐ b. True, the weights of the model are updated with the gradient of the loss function w.r.t. the input
- ☒ c. False, the important quantity is $\nabla_{\mathbf{w}} \mathcal{L}(y, g(\mathbf{x}; \mathbf{w}))$
- ☐ d. False, the update is the loss scaled by the learning rate
- ☐ e. False, the important quantity is the gradient of the weights w.r.t. the inputs

Compute all outputs, preactivations, activations, and delta errors!

Forward pass:



Backward pass:



The hidden units of the forward pass are denoted as s_1 , and s_2 from left to right. The preactivation of the output layer is denoted as s_3 and the output as $\hat{y}$. The delta at the output is denoted as δ_1 , and the hidden deltas as δ_2 , and δ_3 from left to right, respectively. Choose the correct value combination!

Mark all activation functions whose derivative is not continuous!

Wählen Sie eine oder mehrere Antworten:

- ☒ a. ReLU
- ☐ b. Sigmoid
- ☐ c. Tanh
- ☒ d. SeLU
- ☐ e. Linear

Perform the 2D Convolution operation $*$ with a **zero-padding of 1** using the following image $\mathbf{x}$ and kernel $\mathbf{w}$:

$$\mathbf{x} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

and

$$\mathbf{w} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Compute $\mathbf{w} * \mathbf{x}$!

Select one:

- ☐ a. $\begin{pmatrix} 3 & 4 & 3 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{pmatrix}$
- ☐ b. $\begin{pmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{pmatrix}$
- ☐ c. $\begin{pmatrix} 2 & 2 & 3 \\ 2 & 4 & 3 \\ 1 & 2 & 2 \end{pmatrix}$
- ☐ d. (4)
- ☐ e. $\begin{pmatrix} 2 & 2 & 2 \\ 2 & 4 & 3 \end{pmatrix}$

You are working with in a simple convolutional network (CNN) with a single kernel. You have to decide whether this network is able to discriminate images of two classes (see Figure below). Images of the negative class show a red ball in the lower half of the image. Images of the positive class show a red ball in the upper half of the image. Concretely, your network looks like this:

First layer:

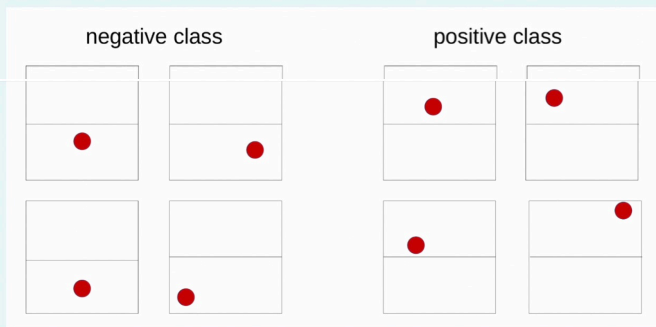
$$\mathbf{h} = \text{ReLU}(\mathbf{W} * \mathbf{x})$$

Second (output) layer:

$$\hat{y} = \sigma\left(\frac{1}{AB} \sum_{a=0}^{A-1} \sum_{b=0}^{B-1} h_{a,b}\right)$$

where $*$ is the standard convolution operation as used in CNNs. An image $\mathbf{x}$ is a standard RGB image that has three channels containing the color intensity values (dimension $[200,200,3]$). The images have a width of A and a height of B and we assume that the feature map $\mathbf{h}$ has the same width and height. σ is the sigmoid function. The first layer is a single convolutional kernel (with a kernel size of 3×3 or 5×5) with adaptive parameters $\mathbf{W}$ that you can freely chose. The second layer performs the so-called *average pooling* operation, that is, is just takes the mean over all neurons.

Is it possible that such a network can discriminate perfectly between the two classes ("red ball in lower half", "red ball in upper half")? In other words: is there a parameter setting for $\mathbf{W}$ that is able to discriminate the two classes?

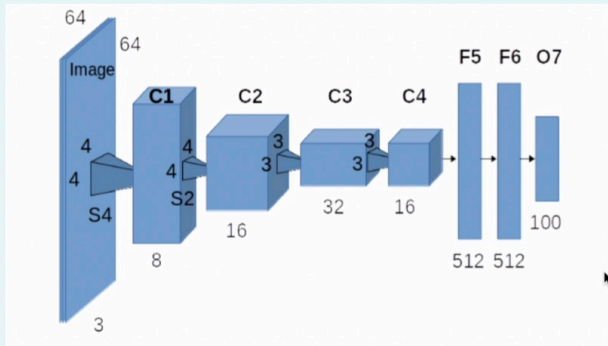


Verble

Wählen Sie eine Antwort:

- ☐ a. Yes, because this is a network with a hidden layer and therefore the universal function approximator theorem applies.
- ☒ b. No, because this network is invariant to the position of the red ball for all parameter settings.
- ☐ c. No, because the max-pooling operation is missing.
- ☐ d. Yes, because gradient descent will find a parameter setting for $\mathbf{W}$ that solves this task.
- ☐ e. Cannot be decided, because it depends on the initialization strategy.
- ☐ f. No, because gradient descent would not converge on such a dataset.
- ☐ g. No, because the ReLU operation makes the network non-linear.
- ☐ h. No, because the sigmoid function is used instead of the linear function at the output unit.

[Meine Auswahl widerrufen](#)



Layer description	Data dimension [width, height, channel] or [#features]	Number of Parameters
Input layer	[64, 64, 3]	0
C1: 4x4 convolutions with 8 kernels, stride 4		
C2: 4x4 convolutions with 16 kernels, stride 2		
C3: 3x3 convolutions with 32 kernels, stride 1		
C4: 3x3 convolutions with 16 kernels, stride 1		
F5: Fully connected layer with 512 units		
F6: Fully connected layer with 512 units		
O7: Output layer, fully connected with 100 units		

You are given a convolutional neural network architecture as in the image above. Your task is to calculate the dimension of an image that is propagated through this network and the number of weights in this network. The initial data dimension is [64,64,3], which corresponds to the width, height, and the

1 Question on the size after last convolution - after C3

1 Question on total number of weights

Assume you have a convolutional layer of 4x4 filters with a total of 16 filters and stride 2 that is applied to inputs of dimensions [32,32,3] (height, width, features) and you want to initialize the weights of this conv-layer according to ~~He's~~ Initialization scheme. Further assume that you want to use a normal distribution. What are the correct parameters for this distribution?

- ☐ a. $W_{ij} \sim \mathcal{N}(0, \frac{\times}{4 \times 4 \times 16})$
- ☐ b. $W_{ij} \sim \mathcal{N}(0, \frac{\times}{4 \times 4 \times 3 + 4 \times 4 \times 16})$
- ☐ c. $W_{ij} \sim \mathcal{N}(0, \frac{\times}{16 \times 3})$
- ☐ d. $W_{ij} \sim \mathcal{N}(0, \frac{\times}{32 \times 32 \times 3 + 32 \times 32 \times 16})$
- ☐ e. $W_{ij} \sim \mathcal{N}(1, \frac{\times}{4 \times 4 \times 16})$

15) Given a fixed NN and a pre-activation

The distribution of the pre-activation s across inputs is near normal distribution with μ and σ^2

Thus $s \sim N(\mu, \sigma^2)$

We apply dropout with

$$\hat{s} = 1/p * d * s$$

d is a Bernoulli random variable with a success rate of p

Multiple Choice:

- $E(s) = \mu$
- $E(\hat{s}) = \mu$
- $\text{Var}(\hat{s}) = \mu^2 * (1/p-1) + 1/p * \sigma^2$
- $\text{Var}(\hat{s}) = 1/p * \sigma^2$
- $\text{Var}(\hat{s}) = p * \sigma^2$
- $E(d^2) = \text{something}$
- $\text{Var}(d) = \dots$
- ...
-

16) A Neural Network has 6 weights, $w = (\dots$ “there were 6 values here”...)

The diagonal elements of the Hessian, $H = (\dots$ ”6 values”...)
Off diagonal elements are close to 0.

Questions on OBD & OBS

- something about magnitude based criterion which to prune first
- With OBS ... w_1 first
- ...
- With OBD ... w_6 first
- ...
- (Can't remember the rest but some variations and questions)

AdaGrad adapts the step size by remembering past gradients $\mathbf{g}^{(j)}$. The parameter wise learning rates are then scaled down for parameters that already experienced large gradients and up for parameters that experienced low gradients so far. For this end, we store the sum of squared past gradients $\mathbf{v}^{(j)} = \sum_{i=1}^j (\mathbf{g}^{(i)})^2$, where the squared is computed component wise. We use this to update the parameter vector $\mathbf{w}$ similar to the basic gradient descent algorithm. Select the correct formula that AdaGrad uses for this update.

Time left 1:26:59

Select one:

- ☐ a. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(i)}} + \epsilon} \odot \mathbf{g}^{(i)}$
- ☐ b. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(i-1)}} + \epsilon} \odot \mathbf{g}^{(i)}$
- ☐ c. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(i-1)}} + \epsilon} \odot \mathbf{g}^{(i)}$
- ☐ d. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(i)}} + \epsilon} \cdot \mathbf{g}^{(i)}$
- ☐ e. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(i-1)}} + \epsilon} \cdot \mathbf{g}^{(i)}$
- ☐ f. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(i)}} + \epsilon} \odot \mathbf{g}^{(i)}$
- ☐ g. $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(i)}} + \epsilon} \odot \mathbf{g}^{(i)}$

18) If we have a training error close to 0 in a Neural Network, how do we choose the best model that generalises?

- validation error small
 - ...
 - Training error is less than the validation error
 - ...
- (• Can't remember the rest)

After optimization of the hyperparameters and architecture on a validation set, the loss on this validation set is an unbiased estimator of the generalization error, True or False?

Wählen Sie eine Antwort:

- ☐ a. False, since the hyperparameters were tuned on the validation set, which introduces bias
- ☐ b. False, because the validation set does not fulfill the i.i.d. requirement
- ☐ c. False, the generalization error is usually estimated by the training set
- ☐ d. True, if the validation set is i.i.d. it is an unbiased estimator of the generalization error
- ☐ e. True, hyperparameter tuning on the validation set does not introduce any bias

20) Multiple Choice on Cross Validation, but none that I have seen in the previous exams